Essence

Abstract

One-Sentence Summarization: "MemSkill reframes agent memory extraction from fixed hand-designed operations into a **learnable, evolvable skill bank** — where a controller selects skills, an LLM executor produces skill-conditioned memories, and a designer evolves the skill set from hard cases."
Contribution: First framework treating **memory operations themselves as learnable and evolvable skills**; introduces the closed-loop trinity of controller (skill selection via RL) + executor (skill-guided memory construction) + designer (skill evolution from hard cases).

Factors

Context

As LLM agents engage in longer, open-ended interactions, they must handle **growing histories** essential for coherence and task performance. Most memory systems rely on **static, hand-designed operations** — fixed primitives (add/update/delete/skip), heuristic modules for extraction/consolidation/pruning — that bake in strong human assumptions about what to store and how to revise memory. These fixed procedures are **rigid under diverse interaction patterns** and scale poorly as histories grow.

Related Work

- **MemoryOS, A-MEM, MemGPT, MemoryBank, ReadAgent**: Static extraction/retrieval pipelines for agent memory
- **Memory-R1, Mem-α**: RL for memory management using downstream task signals, but **fixed memory operation primitives**
- **Evo-Memory, MemEvolve**: Concurrent self-evolving memory work — streaming benchmark and meta-optimization of memory architectures, but **neither targets evolution of memory skills themselves**
- **ExpeL, EvolveR, SkillWeaver, ADAS, AlphaEvolve**: Broader self-evolving agent systems

Gap

All prior memory management approaches rely on **static, hand-crafted operation primitives** — the memory behaviors are fixed at design time and cannot adapt as task demands shift. No prior work treats the **memory operations themselves** (the procedures governing when and

how to extract, consolidate, and prune) as first-class learnable entities. The field **conflates two orthogonal problems**: what to remember (memory content) and how to remember it (memory skill).

Proposal

MemSkill introduces a **skill bank** containing structured memory operations (each with Purpose, When-to-use, How-to-apply, Constraints). A **controller** trained via RL selects a Top-K subset of relevant skills conditioned on the current text span. An **LLM executor** applies the selected skills in a single generation pass to produce **skill-conditioned memories** (span-level processing, not turn-by-turn). A **designer** periodically reviews hard cases and proposes skill refinements or new skills, creating a **closed-loop optimization** over both the skill-selection policy and the skill set itself.

Figures

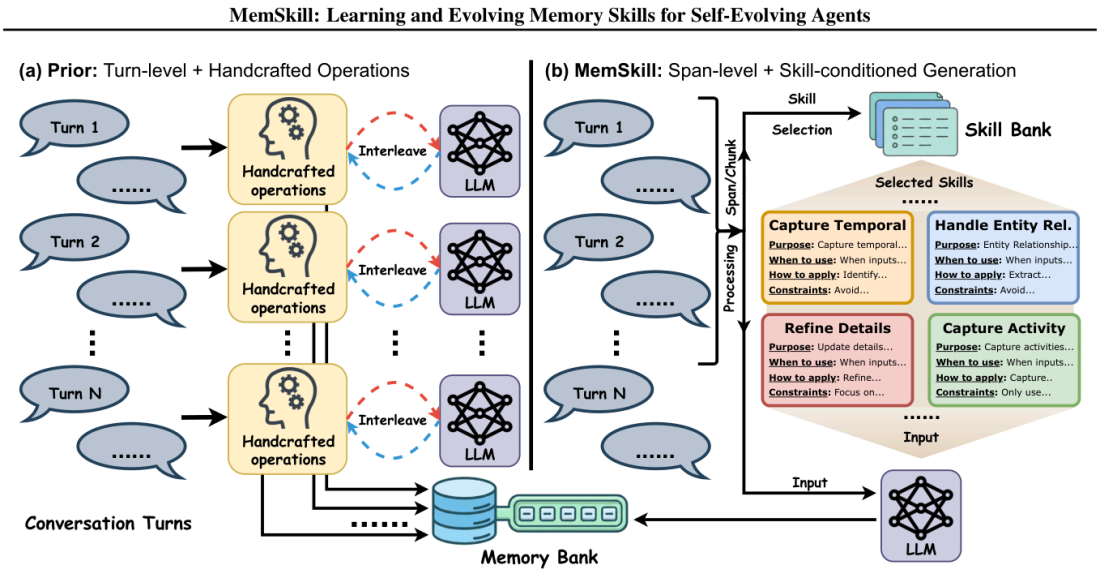

Figure 1. **Comparison between (a) prior turn-level, handcrafted operations and (b) MemSkill's span-level, skill-conditioned generation.** Prior methods interleave handcrafted operations with LLM calls to incrementally extract and revise memory turn by turn, while MemSkill selects a small set of skills from a shared skill bank and applies them in one pass to produce skill-guided memories.

> Architectural shift from **handcrafted turn-level operations** to **skill-conditioned span-level generation** — prior methods interleave fixed operations with LLM calls per turn; MemSkill selects skills from a shared bank and applies them in one pass on larger text spans.

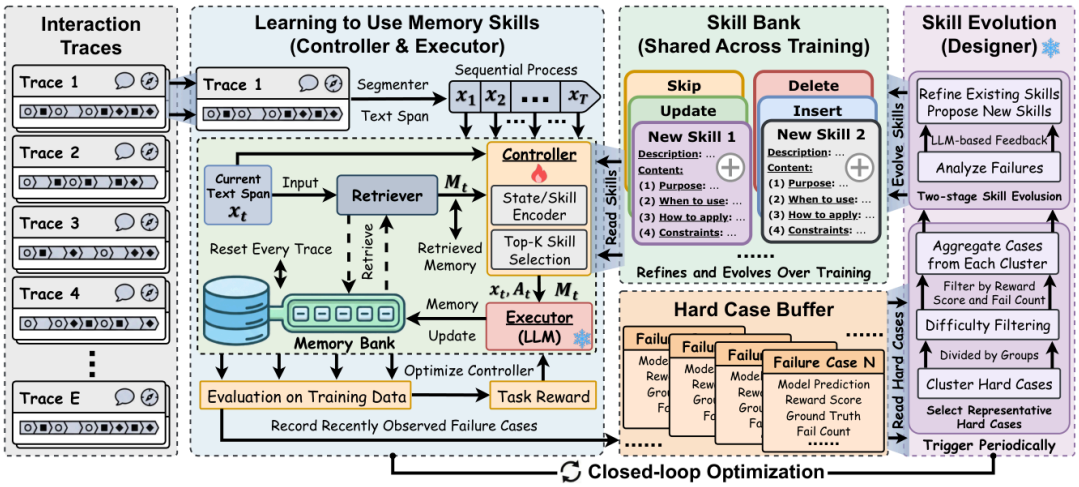

Figure 2. **MemSkill architecture overview.** Given an interaction trace, MemSkill processes it span by span: the controller selects a Top-K subset of skills from a shared *skill bank* conditioned on the current text span and retrieved memories, and an LLM executor applies the selected skills in one pass to update the trace-specific *memory bank*. The constructed memory is then evaluated on memory-dependent training queries to provide task reward for optimizing the controller, while query-centric failures are logged into a sliding hard-case buffer. Periodically, the designer mines representative hard cases to refine existing skills and propose new ones, yielding alternating phases of skill usage and skill evolution. More skill case study can be found in Section 4.5 and Appendix B.

Full system: interaction trace → span segmentation → **controller skill selection** (Top-K) → LLM executor memory construction → **skill designer evolution from hard cases**; the closed-loop optimization cycle is explicitly shown.

Figure 3 shows that MemSkill transfers strongly to HotpotQA across all three context sizes. In particular, MemSkill consistently outperforms strong baselines such as MemoryOS and A-MEM, with the gains becoming more pronounced in the more challenging long-context setting. These results suggest that the learned memory skills are not tied to dialogue-specific surface forms, but capture reusable extraction and revision behaviors that remain effective when the input structure and retrieval demands change.

skills focus on action constraints and object locations, highlighting that embodied success depends on maintaining an actionable world state summary, including task-relevant preconditions rather than broad narrative summaries, to support multi-step execution.

Taken together, these skills illustrate how MemSkill can automatically distill reusable memory behaviors from interaction data and continually refine them through training, moving toward a more adaptive memory system with re-

LoCoMo-trained skill bank transferred to HotpotQA at 3 context lengths (50/100/200 docs): **gains grow at longer contexts**, validating that evolved skills generalize across domains.

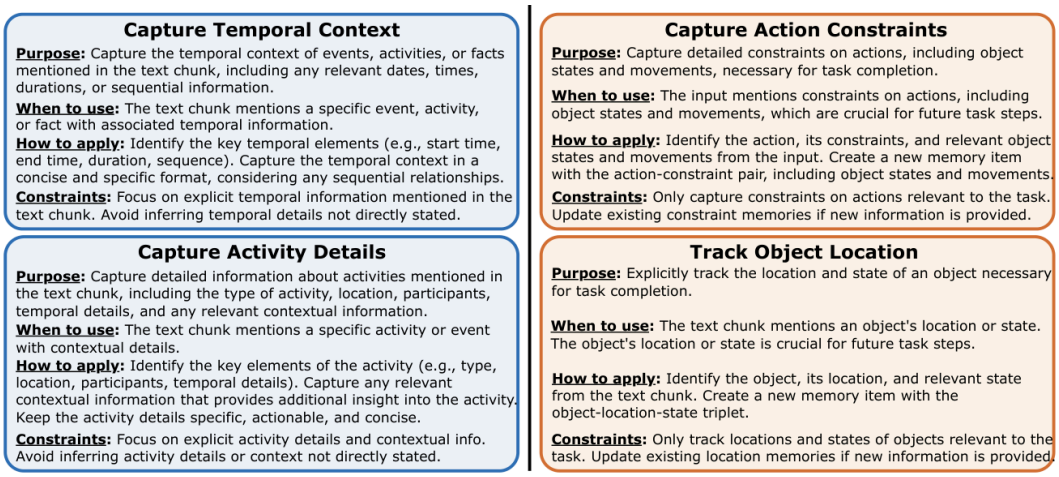

Figure 4. **Case study.** We show representative evolved skills learned on LoCoMo and ALFWorld. ("Description" is omitted for brevity.)

> Concrete examples of **self-evolved memory skill cards** (Purpose, When-to-use, How-to-apply, Constraints) from two domains — shows what the skill bank actually produces.

Note

Figure images live in `raw/compiled/agent-self-evolution/self-evolving-skill-libraries/memskill_figures/`.
Paths are relative from `kb/sources/self-evolving-skill-libraries/memskill.md`.

Critical Analysis

Novel Insight

- **Insight**: Memory operations ("how to remember") are a **distinct and separately evolvable layer** from memory content ("what is stored") — treating them as learnable skills unlocks a new optimization dimension that static systems cannot access.
  - *Prior*: The field treated memory management as a content problem and solved it with fixed procedural logic (yunjue-agent, cascade, skillrl)
  - *Update*: The extraction procedure is itself a variable — **separating the "how to remember" layer from "what is remembered"** enables joint optimization of both
- **Insight**: **Span-level (multi-turn) skill-conditioned memory construction outperforms turn-level interleaved operations** — the granularity of memory extraction has significant downstream impact.
  - *Prior*: Turn-level processing was assumed necessary for capturing fine-grained memory updates (MemoryOS, MemGPT)
  - *Update*: Batching multiple turns into a span allows skills to **reason about temporal relationships across turns**, producing more coherent memories; more frequent extraction is not always better

Fundamental Limitations

- **Limitation**: The skill designer evolves skills based on hard cases but **has no mechanism to detect skill redundancy or contradiction** — skill bank bloat and inconsistency can grow over time.
  - *Root cause*: Skills are evaluated by utility on individual hard cases, **not by coherence with the full skill bank**; the designer lacks a global consistency check
  - *Also affects*: skillfoundry (merge/prune partially addresses this), yunjue-agent (Absorbing Mechanism addresses tool-level redundancy)
  - *Implication*: Self-evolving skill banks risk growing unwieldy — **a skill bank maintenance mechanism** (analogous to SkillFoundry's prune/merge) is necessary
- **Limitation**: The RL-based controller learns skill selection for **specific interaction distributions** (LoCoMo, LongMemEval); transferred to new domains, the selection policy may be suboptimal even if skills generalize.
  - *Root cause*: Skill selection policies **encode task-specific priors** about which skills are likely to be useful — they are not distribution-free
  - *Also affects*: skillrl (SkillBank retrieval policy also encodes training distribution biases)
  - *Implication*: Generalization results (Figure 3) may reflect **skill content generalization rather than controller generalization** — the controller may be a hidden bottleneck in cross-domain deployment

Research Frontier

- **Direction**: **Joint optimization of memory skill content and skill selection policy** in a single training objective, rather than alternating controller RL and designer evolution as separate phases.
  - *Prerequisite*: A differentiable or REINFORCE-compatible representation of skill content that allows gradients to flow from downstream task performance back to the skill designer
  - *Closest attempt*: Memory-R1 (Yan et al., 2025) optimizes memory management via RL but **with fixed operation primitives** — adding skill content as a learnable variable would be the natural extension
- **Direction**: **Skill bank sharing across agent memory systems** — allowing independently trained MemSkill instances to share evolved skills via a compatibility registry.
  - *Prerequisite*: A formalized skill representation with typed applicability conditions for automated compatibility checking
  - *Closest attempt*: evoskills demonstrated cross-model skill transfer for task execution; **the analogous transfer for memory skills is unexplored**

Feeds

- self-evolving-skill-libraries: integrated (subtopic: memory-skill-evolution)
- self-evolving-memory-architectures: integrated (subtopic: skill-conditioned-memory)

Relations

Temporal context: MemSkill (2026-02) applies the self-evolving skill paradigm established by cascade (2025-12) specifically to the memory management domain, arriving simultaneously with skillrl and s1-nexusagent in February 2026. It uniquely occupies the **intersection of memory systems and skill library literature**, importing skill abstraction into a domain where only handcrafted operations existed.

- **cascade** (*extends*): CASCADE applies skill acquisition to **external tool mastery**; MemSkill applies the same paradigm **internally to memory operations** — same learning principle, different target capability layer
- **skillrl** (*complements*): SkillRL evolves **task execution skills** via RL co-evolution; MemSkill evolves **memory extraction skills** via RL controller + LLM designer — orthogonal skill domains sharing the closed-loop optimization pattern
- **yunjue-agent** (*contrasts_with*): Yunjue evolves **executable Python tool primitives**; MemSkill evolves **abstract memory operation specifications** (natural-language structured skills) — executable code vs. declarative skill descriptions
- **evoskills** (*contrasts_with*): EvoSkills generates **multi-file executable skill bundles** via co-evolutionary verification; MemSkill generates **structured memory skill descriptions** via RL + LLM designer — different skill types and evolution mechanisms

Transferable Inspirations

- The **controller+executor+designer trinity** (select skills, apply skills, evolve skills from failures) is a clean three-component architecture for any self-evolving system where selection, application, and refinement are separable concerns
- **Span-level vs. turn-level processing** is a generalizable design choice: larger input granularity enables cross-turn reasoning at the cost of latency; the optimal granularity is task-dependent and should be learned, not fixed
- The **hard-case-triggered evolution mechanism** (designer only activates on failures) is an efficient alternative to continuous evolution — it focuses improvement on the highest-impact failure cases

Cognitive Shifts

- [2026-04-08] MemSkill reveals that "what to remember" and "how to remember" are two independent optimization problems that have been conflated in prior memory systems — separating them enables a new research direction in self-evolving memory. (source: ingest)

Open Questions

- How large can the skill bank grow before Top-K selection becomes a bottleneck — is there an optimal skill bank size beyond which performance plateaus?
- Can the evolved skill cards (natural-language descriptions) be automatically converted to executable memory procedures, bridging MemSkill's declarative skills with SkillRL's executable heuristics?
- How does MemSkill interact with the base LLM's in-context learning ability — do better base models require fewer evolved skills because they can implicitly apply appropriate memory strategies?
- What is the minimum hard-case rate that justifies triggering designer evolution vs. the computational cost of LLM-based skill refinement?